# Deep City Noise

*Feature extraction techniques for audio signals*

Alessandro Di Girolamo

Project repository: https://github.com/aledigirm3/deep-city-noise

**Keywords**: Deep Neural Network, Compuret Science, Spectrogram, Mel Spectrogram, Mel-Frequency Cepstral Coefficients, Audio analysis

**Abstract**

This work explores Urban Sound Classification (USC) through a comparative analysis of different feature extraction techniques for training Convolutional Neural Networks (CNNs). Using the UrbanSound8K benchmark dataset, the main objective is to determine which two-dimensional representation of the audio signal maximizes model performance.

Four main feature extraction techniques were implemented and evaluated: the Short-Time Fourier Transform (STFT), the Log-Spectrogram, the Mel-Spectrogram, and the Mel-Frequency Cepstral Coefficients (MFCCs).

The employed CNN architecture consists of sequential convolutional blocks with Batch Normalization and Dropout for regularization, followed by an Adaptive Average Pooling layer to ensure dimensional adaptability and an MLP classifier. The project adopts a rigorous validation protocol based on 10-fold cross-validation to ensure a robust and unbiased estimation of performance.

The results demonstrate a clear superiority of representations based on the Mel scale. Specifically, Mel-Frequency Cepstral Coefficients (MFCCs) achieved the best performance, reaching an average accuracy of 51% and a macro-average F1-score of 0.53, highlighting that feature compaction and decorrelation are crucial for this classification task.

## 1. Introduction

Urban Sound Classification (USC) is a research area within computational audio analysis that aims to identify and categorize environmental sounds present in urban contexts. The applications of this technology are numerous, ranging from traffic and noise pollution monitoring, to the creation of intelligent surveillance systems, and the development of "smart cities" capable of dynamically reacting to events in their environment.

This document describes a deep learning project for urban sound classification using the benchmark dataset UrbanSound8K[1]. The goal is to understand which feature extraction method is most effective when training a Convolutional Neural Networks (CNNs).

The project is distinguished by a robust methodological approach, which includes:

- The exploration and comparison of various feature extraction techniques from the audio signal to transform it into a 2D representation suitable for CNN analysis.
- The implementation of a modern convolutional neural network architecture, equipped with mechanisms for training stabilization and overfitting prevention.
- The adoption of a rigorous validation protocol based on 10-fold cross-validation to ensure a reliable and unbiased estimation of the model's performance.

The entire pipeline, from data preparation to training and evaluation, has been implemented using the PyTorch framework and its high-level library, PyTorch Lightning, which promotes code modularity and reproducibility.

## 2. Related Works

Environmental Sound Classification (ESC) is an active research area with applications ranging from smart city monitoring to acoustic surveillance and biodiversity analysis. The dataset used in this study, **UrbanSound8K** [1], has become one of the standard benchmarks for validating new models, thanks to its clear class divisions and its 10-fold cross-validation structure, which we have adopted in this work.

Historically, approaches to audio classification relied on traditional machine learning techniques such as Gaussian Mixture Models (GMMs) or Support Vector Machines (SVMs), which were fed with hand-crafted features like MFCCs. However, the advent of deep learning has revolutionized the field.
The primary inspiration was drawn from the success of **Convolutional Neural Networks (CNNs)** in computer vision. Researchers realized that time-frequency representations of audio, such as spectrograms, could be treated as images, allowing CNNs to automatically learn hierarchies of acoustic features. One of the seminal works in this area is by **Piczak** [2], who demonstrated the effectiveness of a simple CNN architecture applied to log-Mel spectrograms extracted from the UrbanSound8K dataset, achieving results that far surpassed previous approaches. Our model, while having its own specific architecture, follows this line of research by adopting an approach based on standard convolutional blocks.

[1] UrbanSound8K dataset

Since then, the scientific community, also driven by competitions like **DCASE (Detection and Classification of Acoustic Scenes and Events)**, has explored more complex architectures. Models like VGGish [3], based on the well-known VGG architecture, and ResNet-based architectures like **PANNs (Pre-trained Audio Neural Networks)** [4] have set the state-of-the-art, often leveraging transfer learning from large audio datasets.

The present work does not aim to compete with these state-of-the-art (SOTA) architectures. Instead, it focuses on a fundamental and crucial analysis: the impact of the choice of input representation. While more complex works take the use of log-Mel spectrograms for granted, our study re-examines this choice by systematically comparing it with alternatives like STFT, log-spectrograms, and MFCCs to quantify the precise contribution of each preprocessing stage on a controlled CNN architecture.

# 3. Data Management and Pre-processing

Proper data management and preparation are a fundamental pillar for the success of any machine learning project. In this case, the UrbanSound8K dataset was used, following a well-defined pipeline for loading, splitting, and transforming the data.

## 3.1 The UrbanSound8K Dataset

The UrbanSound8K dataset is a de facto standard for environmental sound classification tasks. It consists of 8,732 audio clips, each with a maximum duration of 4 seconds. The clips are divided into **10 classes**:

- air_conditioner
- car_horn
- children_playing
- dog_bark
- drilling
- engine_idling
- gun_shot
- jackhammer
- siren
- street_music

A key feature of the dataset is its predefined split into 10 folds. This structure was leveraged to implement a cross-validation strategy, as described in the *train.py* script.

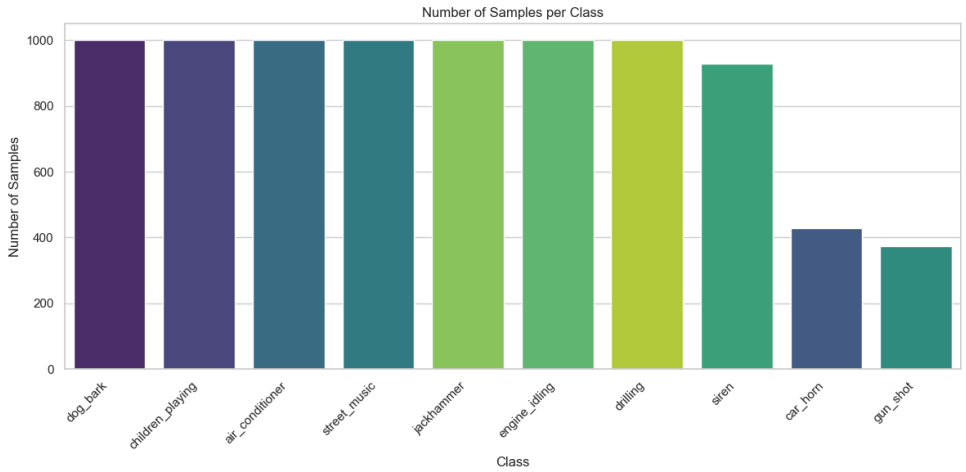


Figure 1: Number of samples per class

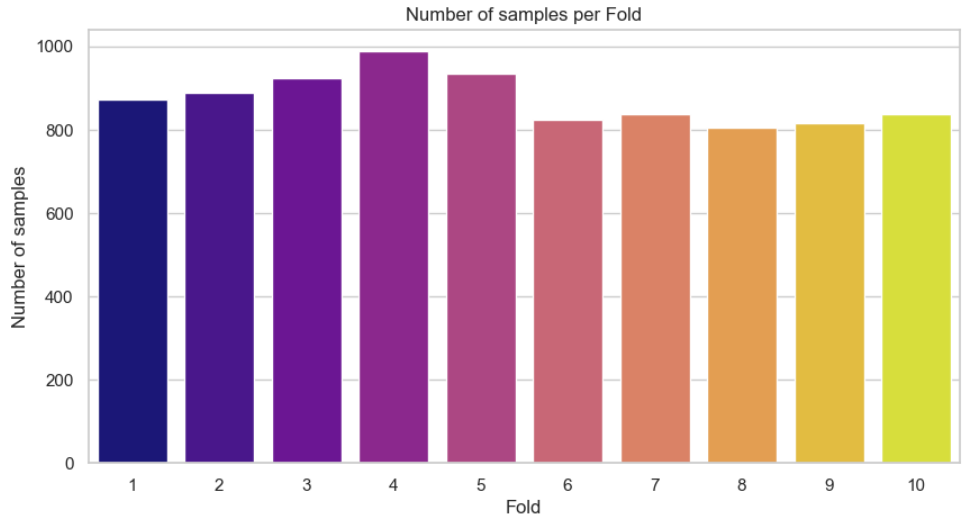


Figure 2: Number of samples per fold

## 3.2 Data Pipeline Structure

Data management is orchestrated by the *UrbanSoundDataModule* class, which encapsulates all the preparation logic according to PyTorch Lightning's best practices.

**Data Splitting (Cross-Validation):** The training process implements a 10-fold cross-validation. For each "fold" (from 1 to 10):

- The Test Set consists of all samples belonging to the current fold.
- The remaining 9 folds are aggregated and used for training and validation.
- This aggregated set is further split: 90% of the data (configurable via *VALIDATION_SPLIT_RATIO*) forms the Training Set, while the remaining 10% constitutes the Validation Set.

This split is performed in a stratified manner, ensuring that the class distribution is kept

constant in both the training and validation sets. This prevents imbalances that could bias the model's learning.

### 3.3 Audio Signal Standardization

Since audio files can have different durations and sampling rates, it is essential to standardize them before feature extraction.
The following steps are performed for each audio file:

- **Resampling:** Each audio file is resampled to a uniform sampling rate of 22050 Hz
- **Mono Conversion:** Stereo signals are converted to mono by averaging the channels.
- **Duration Standardization:** all waveforms are normalized to a fixed duration of 4 seconds. Shorter clips are padded with silence, while longer ones are truncated. This ensures the raw input always has a predictable size.

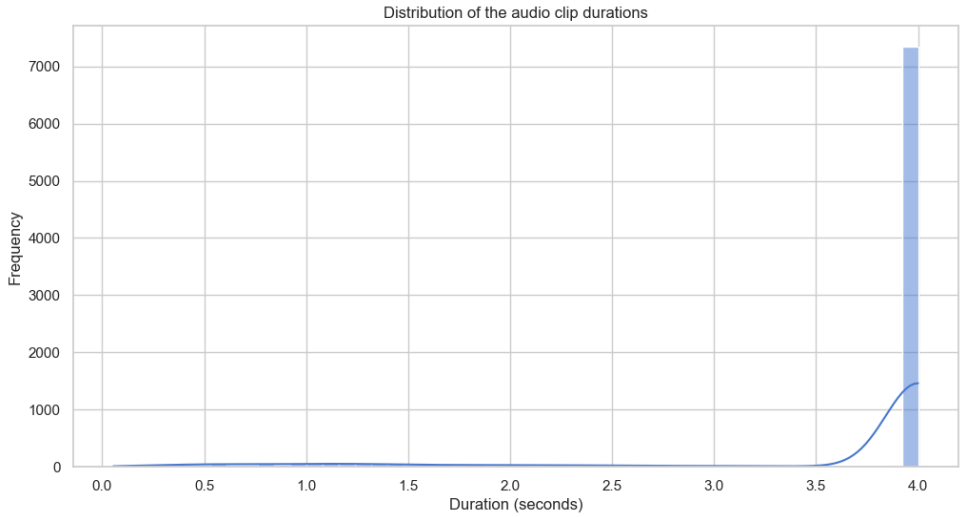


Figure 3: Distribution of the audio clip duration

## 4. Audio Feature Extraction

Convolutional Neural Networks (CNNs) excel at analyzing data with a grid-like structure, such as images. A raw audio signal (a 1D waveform) is not directly optimal for a CNN. The common practice is therefore to transform the audio into a 2D time-frequency representation, similar to an image, known as a **spectrogram**.
The project was configured to experiment with different representations.

## 4.1 Preliminary Human Perception Analysis

Before exploring various feature extraction techniques, it is helpful to introduce some fundamental concepts that aid in understanding the physics of audio signals and human hearing.

First, the amplitude of an audio signal is perceived by the human ear in a way that closely follows a logarithmic trend. For this reason, it is common to convert linear amplitude scales into logarithmic ones (measured in decibels, dB) to better match the way sound intensity is perceived by humans.

A second, equally important, consideration is that the human auditory system is particularly sensitive to amplitude variations in the mid-to-high frequency range, even when the overall signal amplitude is very low.

It is worth referencing two American researchers, Harvey Fletcher and Wilden A. Munson, who in 1933 conducted one of the most influential experiments in the field of psychoacoustics. Their work led to the development of the first equal-loudness curves (also known as isophonous curves), which illustrate how the human ear perceives sound intensity across different frequencies.

In their experiment[2], Fletcher and Munson asked participants to compare pure tones at various frequencies and report how loud they seemed compared to a reference tone, typically set at 1000 Hz. The results showed that the human ear is significantly less sensitive to low and very high frequencies than it is to mid-range frequencies, particularly those between 2 and 5 kHz, where our hearing is most acute.

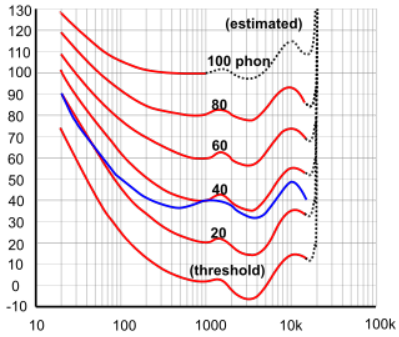


Figure 4: Equal-loudness contours (isophonous curves) as defined by ISO 226:2023

.

## 4.2 Implemented Feature Types

The *UrbanSoundDataset* class uses transformations from the torchaudio library to generate the following features:

- **STFT (Short-Time Fourier Transform):** calculates the Short-Time Fourier Transform, returning the energy distribution of the signal across different frequency

[2] https://en.wikipedia.org/wiki/Equal-loudness_contour

bands over time. Key parameters like *N_FFT* (Fourier window size) and *HOP_LENGTH* (window step size) define its resolution. (Note: The frequency scale is linear (Hz), whereas human hearing is logarithmic).

- **Log-Spectrogram (log_stft):** applies a decibel (dB) conversion to the power spectrogram. This logarithmic scale aligns better with human perception of sound intensity and can help the model better capture amplitude variations.
- **Mel-Spectrogram (log_mel):** a variant of the spectrogram where the frequency axis is mapped to the Mel scale, which mimics the frequency resolution of the human ear (more sensitive to low frequencies). This is often the most effective choice for audio analysis tasks. The number of Mel bands is defined by *N_MELS*.
- **MFCC (Mel-Frequency Cepstral Coefficients):** a set of coefficients that compactly represent the spectrum of a sound. They are calculated by applying a Discrete Cosine Transform (DCT) to a log-Mel spectrogram. MFCCs are effective at capturing the timbral characteristics of sound, decoupling the source from the filter (e.g., the vocal tract). The number of coefficients is defined by *N_MFCC*. (Extremely compact, compression discards a lot of information).

**To view the original waveform and the various spectrograms, see the Jupyter file *'data_analysis.ipynb'*.**

## 4.3 Dimensional Normalization of Features

Even after standardizing the audio duration, small variations due to the transformation algorithms can lead to 2D features with slightly different dimensions on the time axis. To ensure that the input to the neural network is always of a fixed size, an additional padding/truncation step is performed on the extracted feature to match it to an expected time length (*config.EXPECTED_FRAMES*).

## 4.4 Data Augmentation

To improve the model's generalization and reduce overfitting, data augmentation techniques are applied directly to the 2D features (spectrograms). These transformations, **applied only during the training phase**, include:

- **Frequency Masking:** Randomly masks a range of frequency bands.
- **Time Masking:** randomly masks a range of time frames.

These techniques force the model to learn to make predictions even in the presence of partial occlusions in the signal, making it more robust.

# 5. Model Architecture

The core of the system is a Convolutional Neural Network (CNN) architecture, defined in the *CNNClassifier* class of the *model.py* script. The network is designed to process the 2D representations extracted from the audio.

## 5.1 Convolutional Blocks

The architecture is based on the repetition of a standard convolutional block, whose purpose is to extract hierarchies of increasingly complex features. Each block is composed of the following layers:

1. **nn.Conv2d:** a 2D convolutional layer with a 3x3 kernel and "same" padding that applies filters to detect local patterns (edges, textures) in the spectrogram.
2. **nn.BatchNorm2d:** a batch normalization layer that standardizes the output of the previous layer. This significantly stabilizes and accelerates the training process.
3. **nn.ReLU:** a non-linear activation function that introduces the ability to learn complex relationships.
4. **nn.MaxPool2d:** a max-pooling layer that downsamples the spatial dimensions of the feature maps, increasing invariance to small translations and reducing computational load.
5. **nn.Dropout:** a dropout layer that randomly sets a fraction of the outputs to zero, acting as a powerful regularizer to prevent overfitting.

The model stacks three of these blocks, progressively increasing the number of channels (feature maps) from 1 (grayscale input) to 32, then 64, and finally 128.

## 5.2 Input Adaptability and Classifier

A key feature of the architecture is the use of an *nn.AdaptiveAvgPool2d((1, 1))* layer. This adaptive pooling layer computes the average of each feature map and reduces its size to 1x1, regardless of the input spatial dimensions. The result is a fixed-size tensor of shape *(batch_size, 128, 1, 1)*. This makes the model robust to slight variations in input size and simplifies the transition to the classification part.

The final classifier is a fully-connected network (MLP) that receives the flattened feature vector and produces the probabilities for the 10 classes:

1. **nn.Flatten:** flattens the 4D tensor into a 1D vector.
2. **nn.Linear(128, 512):** a dense layer that projects the 128 features into a 512-dimensional space.

3. **nn.ReLU:** a non-linear activation function that introduces the ability to learn complex relationships.

4. **nn.BatchNorm1d, nn.ReLU, nn.Dropout(0.5):** a standard sequence to stabilize the classifier's training and prevent overfitting.

5. **nn.Linear(512, num_classes)** the final output layer that produces the logits (raw scores) for each of the 10 classes.

# 6. Training Phase and Evaluation Strategy

The entire model lifecycle (training, validation, and testing) is managed by the *train.py* script through the use of the PyTorch Lightning framework.

## 6.1 Optimization and Learning

The training logic is defined within the *CNNClassifier* class.

- **Loss Function: Cross-Entropy Loss** is used, which is the standard for multi-class classification problems.

- **Optimizer: Adam** was chosen, an adaptive optimization algorithm that is efficient and generally performs well for a wide range of problems.

- **Learning Rate Scheduler:** to refine the learning process, a *ReduceLROnPlateau* scheduler is employed. This mechanism monitors the *val_loss* (loss on the validation set) and automatically reduces the learning rate if it does not improve for a defined number of epochs (*patience=5*). This allows the model to converge more finely towards an optimal local minimum.

## 6.2 Callbacks for Training Control

To automate and optimize the training process, several PyTorch Lightning callbacks are used:

- **EarlyStopping:** monitors the *val_loss* and stops the training if no improvement is observed for a number of epochs equal to *EARLY_STOPPING_PATIENCE* (10). This prevents overfitting and reduces unnecessary computation time.

- **ModelCheckpoint:** automatically saves the model checkpoint that achieves the best performance on the monitored metric (*val_acc*). At the end of training, the model with the highest validation accuracy is loaded for the testing phase.

- **LearningRateMonitor:** allows tracking the learning rate's progress during training, which is useful for debugging and analyzing the scheduler's behavior.

### 6.3 Testing Methodology

At the end of training for each fold, the best-performing model is evaluated on its respective test set (the held-out fold). The *on_test_epoch_end* function collects all predictions and true labels and prints a detailed Classification Report, which includes key metrics such as precision, recall, and F1-score for each class.
The average accuracy obtained across all 10 test folds is finally calculated to provide a robust and comprehensive estimate of the system's generalization capabilities.

## 7. Results

The evaluation was carried out, as previously mentioned, using cross-validation. Specifically, for each feature extraction method, the results from the 10 folds were collected and the average was computed for each class.
The table below reports the macro-level performance metrics.
A seguire saranno riportati i risultati per ogni tecnica.

### 7.1 STFT (Short-Time Fourier Transform)

| | **Precision** | **Recall** | **F1-score** |
|---|---|---|---|
| **air_conditioner** | 0.067 | 0.0002 | 0.0004 |
| **car_horn** | 0.05 | 0.00 | 0.00 |
| **children_playing** | 0.7650 | 0.5102 | 0.7483 |
| **dog_bark** | 0.934 | 0.078 | 0.144 |
| **drilling** | 0.083 | 0.16 | 0.11 |
| **engine_idling** | 0.43 | 0.14 | 0.21 |
| **gun_shot** | 0.567 | 0.071 | 0.126 |
| **jackhammer** | 0.114 | 0.854 | 0.20 |
| **siren** | 0.127 | 0.017 | 0.03 |
| **street_music** | 0.3 | 0.14 | 0.27 |

Table 1: Average Short-Time Fourier Transform performance metrics

| | **Precision** | **Recall** | **F1-score** | **Accuracy** |
|---|---|---|---|---|
| **macro avg** | 0.26 | 0.23 | 0.11 | 0.15 |

Table 2: Macro average Short-Time Fourier Transform performance metrics

In theory, this transformation does not result in any loss of information from the original signal.
The main drawback is that the human ear does not perceive frequencies on a linear scale. Moreover, the Short-Time Fourier Transform (STFT) allocates the same 'resolution' to all frequency bands, thereby wasting detail in the high-frequency ranges (often less

informative) and compressing the low-frequency ranges, which are crucial for many types of sounds."

## 7.2 Log-Spectrogram (log_stft)

| | Precision | Recall | F1-score |
|---|---|---|---|
| **air_conditioner** | 0.192 | 0.146 | 0.166 |
| **car_horn** | 0.1 | 0.002 | 0.004 |
| **children_playing** | 0.27 | 0.38 | 0.315 |
| **dog_bark** | 0.97 | 0.066 | 0.123 |
| **drilling** | 0.366 | 0.055 | 0.09 |
| **engine_idling** | 0.18 | 0.62 | 0.28 |
| **gun_shot** | 0.00 | 0.00 | 0.00 |
| **jackhammer** | 0.32 | 0.76 | 0.45 |
| **siren** | 0.09 | 0.021 | 0.034 |
| **street_music** | 0.763 | 0.04 | 0.076 |

Table 3: Average log_Fourier Transform performance metrics

| | Precision | Recall | F1-score | Accuracy |
|---|---|---|---|---|
| **macro avg** | 0.32 | 0.21 | 0.154 | 0.24 |

Table 4: Macro average log_Fourier Transform performance metrics

By converting the STFT magnitudes to a logarithmic scale (Decibels, dB), the human perception of loudness is approximated. This compresses the dynamic range of the values, reducing the impact of extremely loud sounds and making details in low-energy parts of the signal more visible.

## 7.3 MEL

| | Precision | Recall | F1-score |
|---|---|---|---|
| **air_conditioner** | 0.24 | 0.545 | 0.333 |
| **car_horn** | 0.752 | 0.25 | 0.38 |
| **children_playing** | 0.782 | 0.51 | 0.617 |
| **dog_bark** | 0.852 | 0.4 | 0.544 |
| **drilling** | 0.58 | 0.46 | 0.513 |
| **engine_idling** | 0.55 | 0.48 | 0.513 |
| **gun_shot** | 0.47 | 0.245 | 0.32 |
| **jackhammer** | 0.46 | 0.58 | 0.52 |
| **siren** | 0.72 | 0.4 | 0.514 |
| **street_music** | 0.65 | 0.8 | 0.72 |

Table 5: Average MEL performance metrics

| | Precision | Recall | F1-score | Accuracy |
|---|---|---|---|---|
| **macro avg** | 0.61 | 0.47 | 0.5 | 0.5 |

Table 6: Macro average MEL Transform performance metrics

The human ear is significantly more sensitive to frequency differences in low-pitched sounds than in high-pitched ones, a standard (linear) spectrogram does not reflect this characteristic of human hearing.
The MEL scale solution warps the frequency axis by stretching it in the lower range and compressing it in the higher range.
Our perception of loudness is also non-linear; therefore, a logarithmic function is applied to the energy values of the Mel-spectrogram. This compresses the dynamic range.

This transformation enabled the model to capture features that are more relevant and closely aligned with human perception, significantly improving performance.

## 7.4 MFCC

| | Precision | Recall | F1-score |
|---|---|---|---|
| **air_conditioner** | 0.453 | 0.472 | 0.46 |
| **car_horn** | 0.675 | 0.25 | 0.36 |
| **children_playing** | 0.72 | 0.524 | 0.61 |
| **dog_bark** | 0.8 | 0.59 | 0.68 |
| **drilling** | 0.63 | 0.41 | 0.45 |
| **engine_idling** | 0.32 | 0.65 | 0.43 |
| **gun_shot** | 0.772 | 0.456 | 0.57 |
| **jackhammer** | 0.45 | 0.75 | 0.56 |
| **siren** | 0.861 | 0.56 | 0.68 |
| **street_music** | 0.86 | 0.34 | 0.49 |

Table 7: Average MFCC performance metrics

| | Precision | Recall | F1-score | Accuracy |
|---|---|---|---|---|
| **macro avg** | 0.66 | 0.5 | 0.53 | 0.51 |

Table 8: Macro average MFCC performance metrics

The Discrete Cosine Transform (DCT) is applied to the log-Mel representation, which is highly effective at separating information and compacting energy. The result of the DCT is a new vector of coefficients, known as cepstral coefficients.
This compression and information separation led to a further improvement in model performance, likely due to the removal of less relevant information that could have introduced confusion.

# 8. Discussions

## 8.1 The Importance of Perceptual Representation

The performance gap between the STFT (15% average accuracy) and features based on the Mel scale (>50% accuracy) unequivocally confirms a fundamental principle of computational audio analysis: alignment with human perception is crucial. The STFT, with its linear frequency scale, treats all frequencies with equal importance. However, as described by the equal-loudness contours (Fig. 4), the human ear does not. A CNN fed with a raw STFT is forced to "waste" computational capacity to learn this non-linearity. The model struggles to distinguish between classes, as demonstrated by the extremely low F1-scores for almost all categories.

The simple switch to a Log-Spectrogram (24% accuracy) provides a tangible improvement. The decibel (dB) conversion compresses the signal's dynamic range, more closely approximating the human perception of loudness and making the features more stable and easier for the neural network to process. However, the real qualitative leap is achieved with the Mel-Spectrogram (50% accuracy). Mapping the frequency axis to the Mel scale reallocates the representation's "resolution," concentrating it in the low-frequency range where most environmental sounds have their distinctive characteristics. This allows the model to capture more robust and discriminative patterns, leading to a doubling of performance.

## 8.2 MFCCs: Effective Compression or Information Loss?

The further, albeit slight, improvement obtained with MFCCs (51% accuracy) is interesting. MFCCs are essentially a compressed and decorrelated version of log-Mel spectrograms, obtained via the Discrete Cosine Transform (DCT). In theory, this process entails a loss of information. Why, then, do they perform better?
A possible interpretation is that, for a relatively simple CNN architecture like the one used, this "pre-digestion" of features is advantageous. The DCT compacts the timbral information into the first few coefficients, removing redundancy and background noise. This may help the model focus on the most salient aspects of the sound, avoiding overfitting on spurious details present in the full Mel-spectrogram.
It is plausible that deeper and more complex architectures (e.g., ResNets) could extract more value from the information-rich Mel-spectrogram, but for our model, MFCCs represent the best trade-off between information density and signal cleanliness.

## 8.3 Class Analysis and Limitations

Analyzing the per-class results (Tables 5 and 7), we note that sounds with well-defined timbral and temporal characteristics, such as *street_music*, *dog_bark*, and siren, achieve

high F1-scores with Mel/MFCC features. Conversely, stationary and broadband sounds like *air_ conditioner* or *engine_ idling* are more difficult to distinguish, likely due to their spectral similarity. The confusion between these two classes is a known issue in the literature.

It is important to acknowledge the limitations of this study. The CNN architecture, while effective for a comparative analysis, is basic. Using techniques like transfer learning from models pre-trained on large audio datasets (e.g., AudioSet) could dramatically improve the absolute performance. Furthermore, only basic augmentation techniques (Time and Frequency Masking) were used. More advanced strategies like Mixup or pitch and time-stretching variations could make the model even more robust.

### 8.4 Future Perspectives

This work opens the door to several future directions. It would be interesting to conduct a more in-depth error analysis using confusion matrices to systematically identify which classes are mistaken for one another.
Additionally, one could investigate whether increasing the model's complexity (e.g., by switching to a ResNet architecture) alters the performance hierarchy of the features, perhaps favoring Mel-spectrograms over MFCCs. Finally, applying the same methodologies to other environmental sound datasets could verify the generalizability of the conclusions reached here.

## 9. Conclusion

In conclusion, this comparative analysis unequivocally demonstrates that the choice of feature extraction technique is a critical factor.
Techniques incorporating human perceptual models, such as the Mel scale, clearly outperform linear representations.
The additional step of compaction and decorrelation provided by Mel-Frequency Cepstral Coefficients (MFCCs) proved to be the most effective, making them the optimal choice for the convolutional neural network architecture used in this study, as they provide an audio signal representation that is both rich in pertinent information and robust to noise.

## References

[1] J. Salamon, C. Jacoby, and J. P. Bello, "A dataset and taxonomy for urban sound research," in *Proceedings of the 22nd ACM international conference on Multimedia*, pp. 1041–1044, ACM, 2014.

[2] K. J. Piczak, "Environmental sound classification with convolutional neural networks," in *2015 IEEE 25th International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6, IEEE, 2015.

[3] S. Hershey, S. Chaudhuri, D. P. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold, *et al.*, "CNN architectures for large-scale audio classification," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 131–135, IEEE, 2017.

[4] Q. Kong, Y. Wang, M. D. Plumbley, and W. Wang, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.